UltraZoom: Generating Gigapixel Images from Regular Photos

UltraZoom is a system for **generating gigapixel-resolution images** of objects using "normal" capturing inputs (i.e. handheld phone photo).
Given a set of close-ups and the full image, the system upscales the full image to match the scale and material details of the close-ups. The final result is a unified, high-resolution image where local details reflect the object's appearance (they are photorealistic and faithful)

Introduction

When we take photo, we would like to have both a full overview and fine-grained details but normal cameras (i.e. camera of the phone) have a **fixed resolution**. So, we have to decide a **trade-off** between coverage and detail that we can obtain.

Example (browse a product online):

The shops usually shows:

- Full View of the objects
- Close-up to show material details

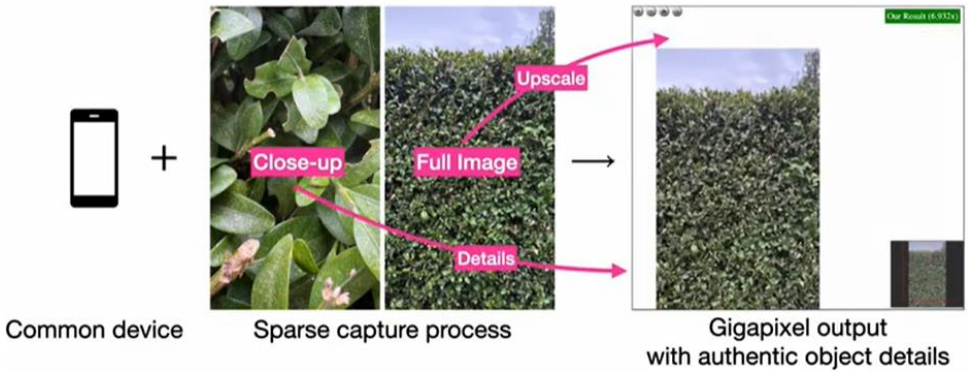
But the **closeup only convers a small region** of objects (no detail elsewhere). For the rest of the object, we have to image the details.

A solution to solve this problem is **Gigapixel image**: digital image composed of billions of pixels. In this way, we can zoom and pan freely to see detail everywhere in the image.

Capturing Gigapixel image isn't easy: it requires a special built camera rig to systematically capture thousands of images across the scene. We would need a specialized camera to densely capture every part of the scene. This is needed for complex scene where each local region has unique details.

Is there a more efficient way to create gigapixel images for everyday objects? **Single objects** often repeat the same material details (it's characterized by a **limited set of material patterns**) which means that with a sparse set of captures (close-up snapshots), it's possible to extrapolate the rest of the object (suffices to **cover the range of textures**). In other world, from these sparse captures, it's possible to infer the missing fine details across the object in a plausible and truthful manner.

Based on this insight, here's our problem formulation: using commonly available device and a simple sparse capture process, our goal is to generate gigapixel images of objects with authentic object details.



More specifically, our goal is to **upscale the full image** to gigapixel resolution by extrapolating material details from the captured close-ups.

Method

The work doesn't assume to have gigapixel images. We construct the system using close-ups images of an object and regular-resolution image showing the entire object. The goal is to upscale the full-view image to the same scale as the close-ups. In this way we produce a gigapixel-resolution output with details faithful to the exemplars and structure aligned with the original full view.

To tackle this problem, we designed a system with three stages:

1) **Dataset construction**: where we capture an object and process the captures into a pair data set.
2) **Per-instance fine-tuning**: where we fine-tune pre-trained generative models on the constructed data set
3) **Gigapixel inference**: where we divide and merge separate inference runs to obtain the gigapixel output.

Dataset construction

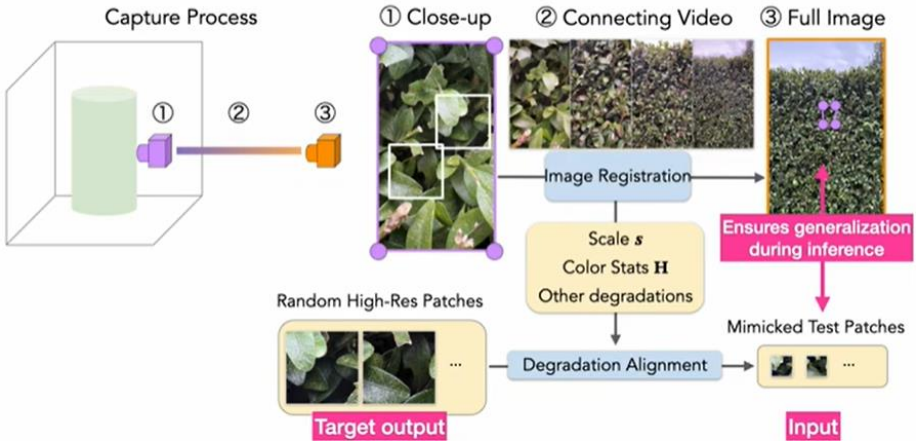
We begin by capturing **close-up**, **full image** and a **video** connecting the two. Using these images the goal is to construct a **paired dataset** for the fine-tuning phase (next phase). However, we can't directly work with the close-ups and the full image to set up the dataset due to important difference between the two types of images:

- Images are taken with different lenses and different distances (large variation perspective)
- Different colour and different level of noise
- Image centre misalignment (caused by handheld capture)

These factors complicate the process of achieving perfect spatial alignment (**pixel-to-pixel correspondences**) between close-up shots and their low-resolution pixels in the full image. To circumvent this problem, pixel-aligned pairs are synthetically generated by taking high-resolution close-ups and applying **controlled degradation** to simulate the full-view image's characteristics.

In other words, we take random patches (Random High-Res Patches) from the close-up and degrade them to mimic the appearance of patches from the full image. The degraded patches and the original patches form input output pairs for per-instance fine-tuning.

Dataset construction schema:



Executing this degradation requires two critical steps:

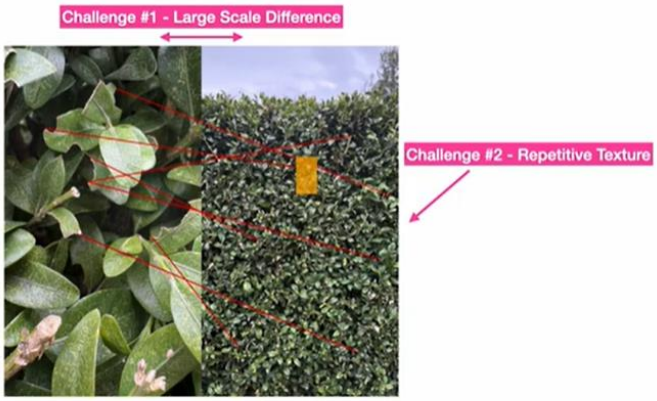
1) Determine the **relative scale** (between the close-ups and the full view) to ensure accurate downsampling.
2) Identify the **spatial region** within the full image to replicate its specific colour and noise profiles (and other degradation characteristics).

Both operations rely on a preliminary stage of **coarse image registration**: it's a fundamental task in image processing that involves aligning two or more images to a common coordinate system. In this work, it isn't used technique like SIFT due to significant differences and characteristics between the close-ups and full view images.

Getting this degradation right is important for the model to generalize at test time. We register the close-up within the full image to derive degradation parameters:

- a skill factor for downscaling.
- colour statistics for colour tone adjustment.
- additional degradations to close the remaining appearance gap.

However, existing image registration approaches fail on our images



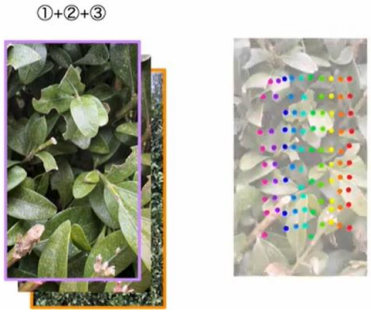
Wrong correspondences ⟶ Wrong registration matrix

We can see that the correspondences are wrong. On the right half, the line should all land in the orange box region, but they're currently scattered everywhere. This is due to two challenges with our data:

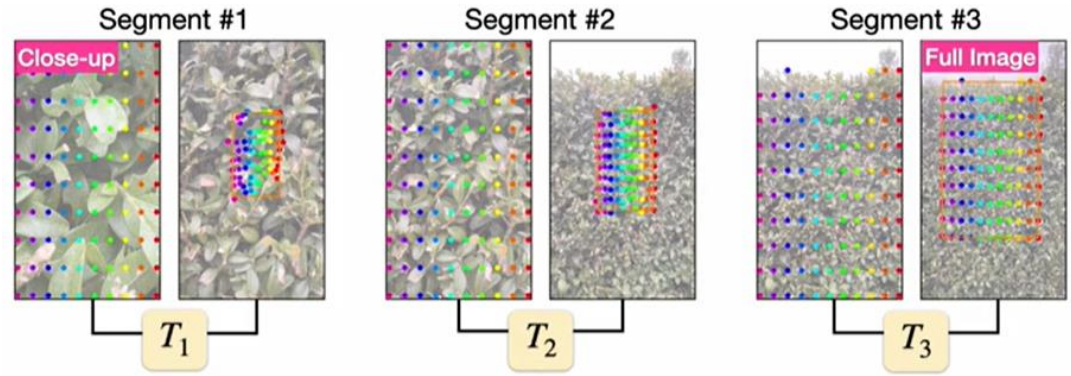
- the extreme scale difference results in different feature vectors for even corresponding content
- the repetitive texture makes feature matching highly ambiguous.

With unreliable correspondences, the resulting registration matrix would also be wrong. Since it's challenging to do direct registration in our approach, we **capture video** to connect the close-up to the full image.

We concatenate the close-up, connecting video and full image into a single video and run a state-of-the-art point tracking method across the video frames. The points are reinitialized every 100 frames to keep the tracking stable as the content changes very quickly.



We show a breakdown of the tracking process. There are three segments separated by the points reinitialization. For each segment, we estimate a 2D similarity transform between the first and last frame using RANSAC.



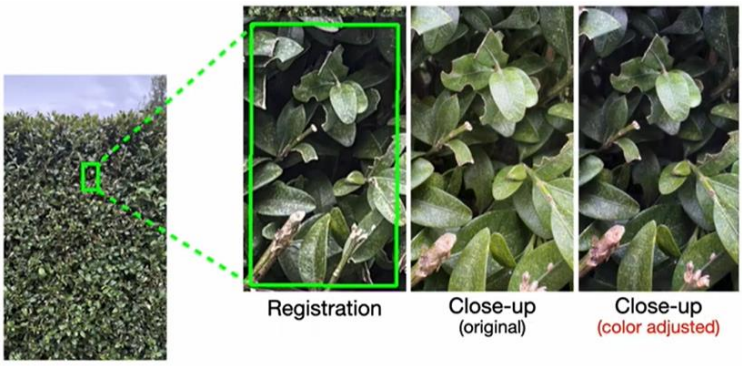
We track the close-up region across the video to register it within the full image (estimating the scale and the colour statistics of the matched region).

By chaining the segment level transforms, we now have the **full transformation** that registers the close-up and the full image. From the registration matrix, we can now **compute the scale factor** needed for image downscaling.

$$T_{C\to F} = T_3 \circ T_2 \circ T_1$$
$$s = \sqrt{\det\left(T_{C\to F[:2,:2]}\right)}$$

The scale s is extracted from the upper-left 2x2 submatrix (corresponds to the rotation and scaling component of the final transform T_C

Now we can also extract colour statistics from the register region to adjust the colour tone of the close-up so that it matches the appearance of the full image.

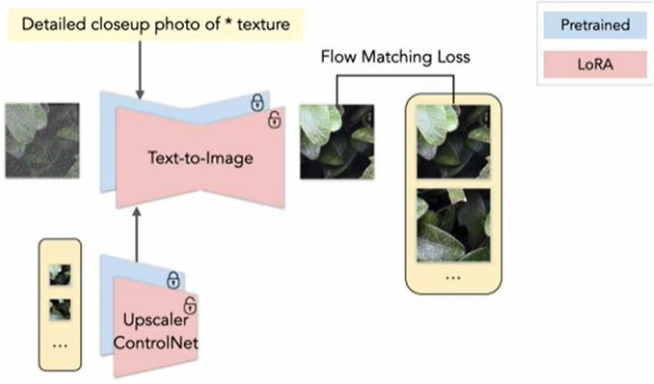


With the constructed peer data set, we now move on to stage two.

Per-Instance Fine-tuning

We fine-tune a separate copy of model weights for each object to make sure the generated details are specific to the object. The model has two components:

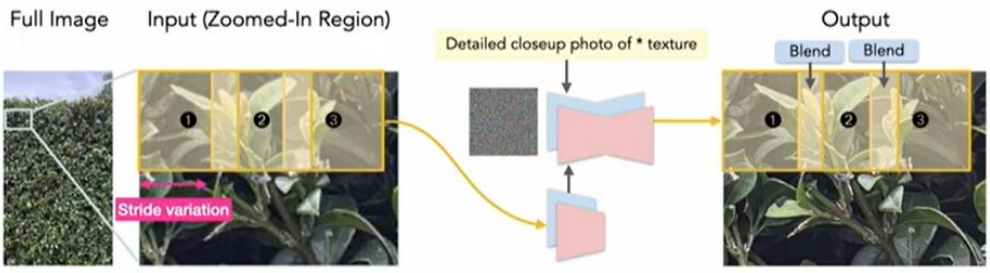
- Text-to-image backbone that takes in a noise image and text prompt
- Super resolution ControlNet that takes in the low-resolution condition.



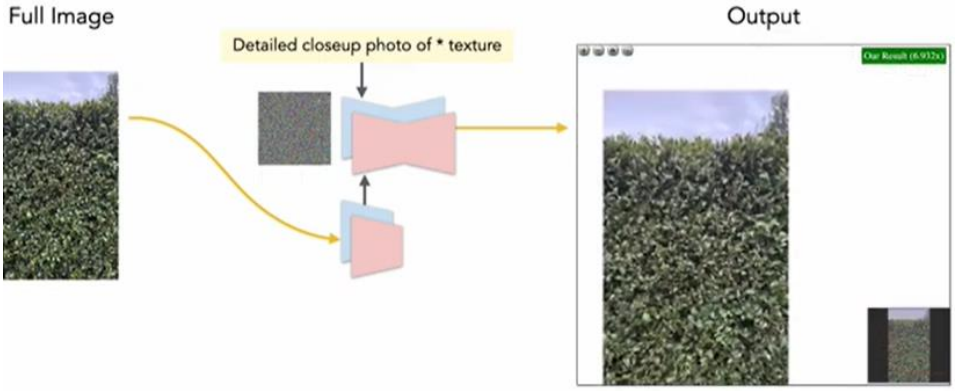
We freeze the pre-trained weights and fine-tune the lightweight low rank adapters (LoRA) using the same flow matching objective used for pre-training. Once the finetuning converges, we move on to stage three, gigapixel inference.

Gigapixel inference

Due to the extremely large output size and limited GPU memory, we split the input into overlapping tiles (as shown) with patches one, two, and three. And the tiles are processed by the model one by one. To make sure there's no boundary artifacts in the final output, we blend the overlapping regions at the end of each denoising step and after decoding to RGB pixels. The overlapping latent regions are averaged. To avoid boundary artifacts, we also vary the stride across the denoising steps to avoid repeating the same tile boundaries.

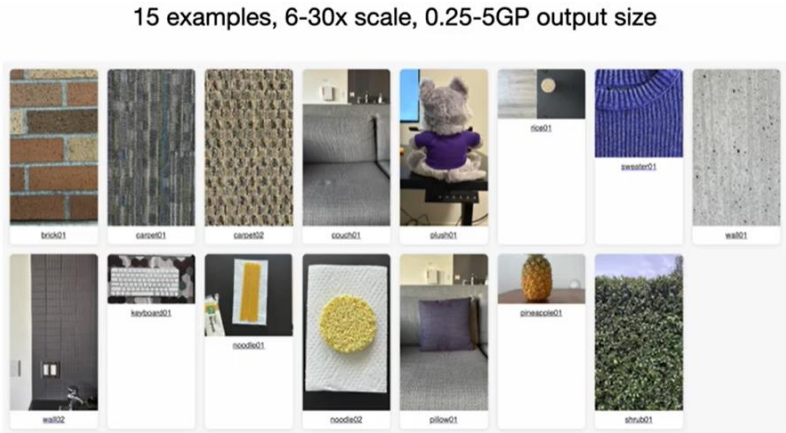


At the end of the stage, we have a seamless gigapixel resolution output that closely approximates a real captured gigapixel image.



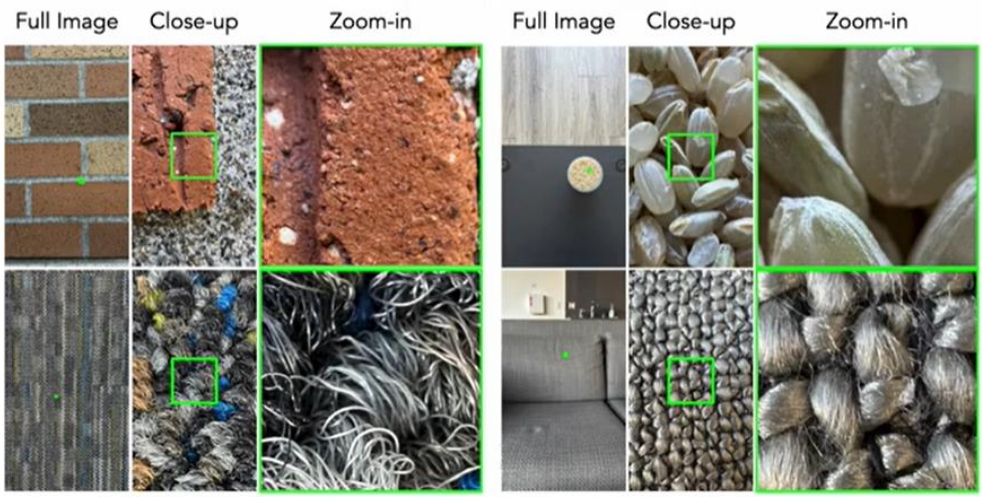
Evaluation data

To evaluate our method, we collected 15 examples in total with scale ranging from 6x to 30x and output resolution ranging from 0.25 to 5 gigapixels.



Here are some examples of the full image and closeup we captured. We capture everything with an iPhone 16. The full images are captured with a regular lens, and the close-ups are captured using the iPhone's built-in macro lens, which allows us to get very close to object surface and capture very interesting looking details.

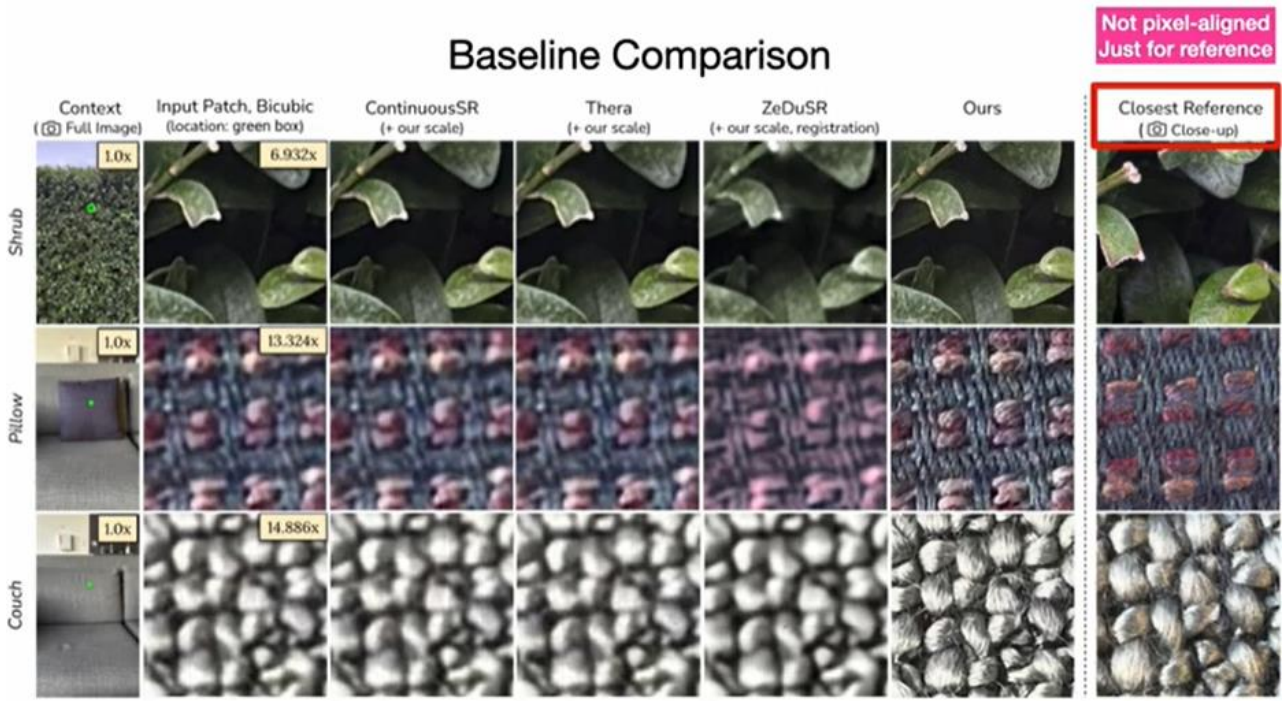
Example captures (iphone 16, regular and macro lens):



Results

I'm playing showcases our gigapixel outputs in an interactive viewer. The top right corner shows the max zoom factor and the bottom right corner shows the current viewing position

Here we compare our method with three baselines. across several examples. Each row corresponds to a different object ordered from low to high scale.



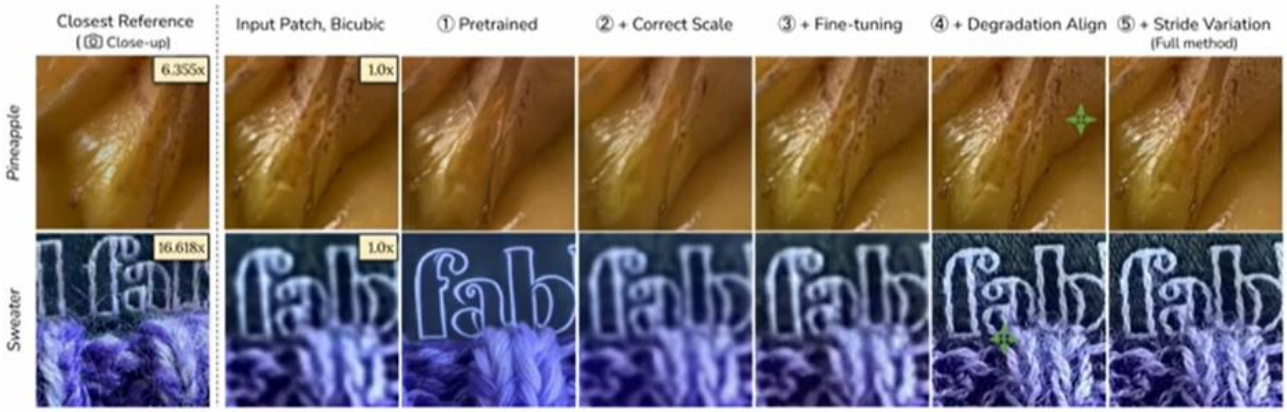
Columns:

- The first two columns show the full view image for context and an input patch coming from the green box and the full image.
- The next four columns show results from the three baselines and our method.
- The final column shows the closest matching reference patch from the close-up image located via our estimated registration.

Note that the input patch and the reference patch are not pixel aligned as they come from separate captures taken at different distances.

Ablation studies

Here we show how each component of our method contributes to the final result using two examples.



Columns description:

- **First two columns**: show the closest reference patch from the close-up and the input patch from the full image. Again, the reference patch is more of a visual reference than pixel aligned ground truth.
- **Pre-trained**: in the most naive version, we run the pre-trained model at the default 4x scale. The output is sharp, but the content drifts and the details don't match the reference.
- **Correct scale**: if we switch to the correct scale without fine-tuning, the output becomes blurry. Since this scale is out of distribution for the pre-trained model.
- **Fine-tuning**: if we add fine-tuning without proper degradation alignment, the outputs look more detailed than version two, but the results still contain artifacts due to the train test domain mismatch.
- **Degradation Align**: once we add a degradation alignment, the train test gap narrows and the quality improves a lot. The patch wise inference without stride variation still leaves visible seams like the faint grid lines indicated by the green arrows.
- **Stride Variation**: finally, our full method addresses all of these issues, producing seamless and faithful gigapixel results.